

OFFICIAL HANDBOOK • V1.0.0

HANDLER

The Complete Guide to CSV Data Auditing, Surgical Patching & Autonomous Remediation

A modern, memory-efficient command-line utility built in Rust. Designed to turn dirty, unreliable tabular data into sanitized, production-ready datasets for humans and autonomous AI agents alike.

4-in-1 Diagnostics

Scans missing values, automatically infers column data types, detects statistical IQR outliers, and finds duplicate records in sub-second speeds.

Surgical Pinpoint Patching

Target exact cells by row index and column name to repair broken data with surgical precision, avoiding destructive blind row deletion.

Bulk Remediation Engine

Deduplicate, coerce messy types, standardize missing tokens, or impute numerical medians automatically with `--fix`.

Autonomous Agent Native

Complete `--json` machine-readable output mode allowing LLMs and automated pipelines to audit, reason, and repair without human intervention.

AUTHOR & MAINTAINER

Snehil Khokhar
(@khokharsnehil45)

IMPLEMENTATION

Rust (Zero-dependency Runtime)

LICENSE

MIT Open Source

GITHUB REPOSITORY

github.com/khokharsnehil45/handler

1. Getting Started (Zero Knowledge Track)

Welcome to **handler**! If you have never used a terminal or command-line tool before, don't worry. This chapter walks you through what a command-line interface (CLI) is, how to install handler in 10 seconds, and how to inspect your very first file.

1.1 What is a Command-Line Tool?

A CLI is a program where you type short instructions (called commands) instead of clicking buttons in a graphical window. CLIs are preferred by software engineers, data scientists, and AI agents because they are:

- **Blazing Fast:** They start instantly without loading heavy graphical interfaces.
- **Memory-Efficient:** They stream files directly from disk without crashing your computer.
- **Scriptable:** You can chain them into automated scripts and scheduled cron jobs.

1.2 Instant 1-Command Installation (Linux & macOS)

You can install `handler` on any Linux or macOS system using standard `curl`. Open your Terminal application and paste the following line:

```
curl -sSL https://raw.githubusercontent.com/khokharsnehil45/handler/main/install.sh | bash
```

✓ What happens behind the scenes:

The installer automatically detects your operating system and CPU architecture, downloads the precompiled binary from GitHub Releases, marks it executable, and places it in your path (`~/local/bin/handler`).

To verify that installation succeeded, run:

```
handler --version
```

You should see `handler 0.1.0` printed to your screen!

1.3 Anatomy of a CSV File

CSV stands for **Comma-Separated Values**. It is the most universal data format in the world. It looks like a plain text grid:

```
user_id,username,age,signup_date,is_active
101,alex_m,25,2026-01-15,true
102,sarah_k,30,2026-02-10,false
103,john_d,twenty-two,2026-02-18,true
```

- **Header:** The very first row defining the names of the columns.
- **Rows:** The individual data entries below the header.
- **Cells:** The individual fields separated by commas.

2. The 4 Diagnostic Audits

Real-world CSV files are notoriously dirty. People misspell words, databases export unformatted dates, sensors output offline codes, and copy-paste errors duplicate rows. `handler` provides 4 specialized audit engines to uncover these flaws.

2.1 Audit 1: Missing Value Detection (`--search_missing`)

Missing data isn't always just an empty cell. In practice, messy files contain various placeholder strings. `handler` recognizes:

- Completely empty cells (`" "`)
- Whitespace-only cells (`" "`)
- Standard placeholder tokens: `NA` , `N/A` , `NaN` , `null` , `None` , `nil` , `-` , `?`

```
handler data.csv --search_missing
```

Example Output:

 Scanning 'data.csv' for missing values...

Metric	Value
Total Rows (Data)	10
Total Missing Values	6
Rows with Missing Values	5 (50.0%)

 Column Breakdown:

Index	Column Name	Missing Count	Missing %	Status
1	user_id	0	0.0%	✓ Clean
2	age	3	30.0%	✗ 3 missing
3	email	2	20.0%	✗ 2 missing

2.2 Audit 2: Automatic Type Inference & Mismatches (`--search_types`)

Because CSV files store everything as text, software often breaks when a number column suddenly contains text. `handler` reads non-empty cells and mathematically infers the dominant expected type using a **≥60% majority rule**.

Supported Type	Pattern Recognized	Example Invalid Value Flagged
Integer	Whole numbers (<code>101</code> , <code>-42</code>)	<code>"twenty"</code> , <code>"35.5"</code> (decimals in integer col)
Float	Decimals & scientific notation (<code>3.14</code> , <code>1e5</code>)	<code>"\$49.99"</code> (unstripped dollar sign)

Supported Type	Pattern Recognized	Example Invalid Value Flagged
Boolean	true , false , yes , no , 1 , 0	"maybe" , "pending"
Date	YYYY-MM-DD or DD-MM-YYYY	"yesterday" , "2026/99/99"
Text	Fallback for arbitrary strings	Always clean (all strings valid)

```
handler data.csv --search_types
```

Every mismatched cell is reported with its exact **Row Number**, **Expected Type**, **Offending Value**, and **Clear Reason**.

2.3 Audit 3: Statistical Outlier Detection (`--search_outliers`)

An outlier is a number that is technically valid, but statistically abnormal (e.g. `age = 240` or `salary = $9,500,000`). `handler` uses the industry-standard **IQR (Interquartile Range) Method**:

- 1

Sort and Find Percentiles
Computes the 25th percentile (Q1) and 75th percentile (Q3) of all valid numbers in the column.
- 2

Compute IQR Span
IQR = Q3 - Q1 (the middle 50% spread of your data).
- 3

Establish Statistical Fences
Lower Fence = `Q1 - 1.5 × IQR` | Upper Fence = `Q3 + 1.5 × IQR` .

```
handler data.csv --search_outliers
```

Example Output:

✓ Outlier Detection Report (Method: IQR):

Index	Column Name	Inferred Type	IQR Expected Range	Min / Max	Outliers	Status
3	age	Integer	[16.56, 49.06]	25 / 240	1	* 1 outlier(s)
4	salary	Integer	[34125, 77125]	48000 / 950000	1	* 1 outlier(s)

⚠ Outlier Details:

Row #	Column	Value	IQR Valid Range	Reason
10	age	240	[16.56, 49.06]	Exceeds upper bound 49.06 (+25.0x IQR)

2.4 Audit 4: Duplicate & Collision Detection (`--search_duplicates`)

Duplicates cause inflated metrics, false customer counts, and database primary key errors. `handler` supports two distinct scoping modes:

Mode A: Full-Row Identical Duplicates

Checks if every single column in row A is byte-for-byte identical to row B:

```
handler data.csv --search_duplicates
```

Mode B: Key-Column Collision Detection (`--key <COLUMN>`)

Identifies rows that share the same unique identifier (such as `user_id` or `Email`), even if their other columns differ:

```
handler data.csv --key Email
```

 Duplicate Detection Report:

Duplicate Metric	Value
Scope	Key Column 'Email'
Total Rows	12
Unique Records	10
Duplicate Rows	2 (16.67%)
Duplicate Clusters	2

3. The Full Audit Command

Instead of running four separate commands, running `handler` directly on a CSV file automatically executes **all four audits** in a unified pass:

```
handler data.csv
# or explicitly:
handler data.csv --audit
```

💡 Inspecting all rows with `--verbose`

By default, `handler` prints the first 10 offending rows to prevent overwhelming your terminal. When inspecting long reports, pass `--verbose` (or `-v`) to view every single affected row without truncation:

```
handler data.csv --verbose
```

4. Surgical Pinpoint Patching (Precision Editing)

The greatest danger in data cleaning is **blind deletion**. Throwing away an entire row because one cell has a typo causes significant data loss. `handler` introduces **Surgical Pinpoint Patching**.

4.1 Syntax: `--patch "ROW:COL=VALUE"`

You can target any cell by its 1-based Row Number and Column Name:

```
handler data.csv --patch "3:age=22" --output clean.csv
```

Or target by 1-based Column Index:

```
handler data.csv --patch "3:3=22" --output clean.csv
```

4.2 Patching Multiple Cells Simultaneously

You can pass multiple `--patch` arguments in a single execution:

```
handler data.csv \
  --patch "3:age=22" \
  --patch "5:signup_date=2026-03-01" \
  --patch "7:is_active=true" \
  --output clean.csv
```

4.3 Surgical Row Dropping (`--drop-row`)

If a specific row is completely unsalvageable (e.g. Row 10 contains an impossible outlier), drop that single row:

```
handler data.csv --drop-row 10 --output clean.csv
```

4.4 In-Place Editing (`--in-place`)

When modifying production data, you can apply edits directly to the source file without managing a second file.

`handler` writes to a temporary file first and performs an **atomic rename** to ensure zero risk of file corruption:

```
handler data.csv --patch "3:age=22" --in-place
```

5. Bulk Auto-Remediation (`--fix`)

When dealing with hundreds of dirty rows, manual patching is impractical. The bulk remediation engine (`--fix`) applies automated sanitation strategies to export a clean dataset.

5.1 The Smart Auto-Repair Preset

Running `handler data.csv --fix` automatically applies proven data-cleaning standards:

```
handler data.csv --fix --output clean.csv
```

- **Deduplication:** Automatically drops duplicate rows, keeping only the first valid occurrence.
- **Type Coercion:** Coerces floats to whole numbers in integer columns (e.g. `35.5` → `36`) and normalizes boolean variants (`yes` → `true`).
- **Token Standardization:** Converts messy placeholders (`NA` , `null` , `None`) into clean empty strings (`" "`).
- **Drop Fatal Errors:** Drops rows containing unparseable strings in numeric or date columns (e.g. `"twenty-two"` in an age column).

5.2 Granular Repair Strategies

Customize the remediation behavior to match your pipeline's exact requirements:

Command Flag	Remediation Action	Best Used When...
<code>--fill-missing</code>	Imputes missing numeric cells with the column's statistical median .	Training ML models where nulls are rejected but dropping rows loses training data.
<code>--drop-missing</code>	Drops any row containing at least one missing cell.	Strict pipelines where synthetic imputation is forbidden.
<code>--drop-outliers</code>	Drops rows containing values outside the IQR fences.	Removing test artifacts, sensor glitches, or extreme fat-finger typos.
<code>--drop-duplicates</code>	Deduplicates rows or unique keys.	Ensuring single-customer records for marketing or billing systems.

Example: Impute missing values with medians and drop outliers:

```
handler data.csv --fix --fill-missing --drop-outliers --output final_dataset.csv
```

Remediation Report Preview:

 CSV Repair Report:

Repair Metric	Value
Input File	data.csv (12 rows)
Output Clean File	final_dataset.csv (10 rows)
Total Rows Dropped	2 (16.7%)
- Duplicate Rows Dropped	1

- Outlier Rows Dropped	1
Cells Imputed (Filled)	1 (median filled)
Values Coerced (Standardized)	2
Export Status	✓ Clean CSV Exported Successfully

6. AI Agent Integration & Architecture (`--json`)

Large Language Models (LLMs) like GPT-4, Claude, and Gemini are powerful reasoning engines, but they perform poorly when processing raw CSV text directly in their context window:

- **Context Window Waste:** Loading a 50,000-line CSV consumes hundreds of thousands of tokens.
- **Counting Hallucinations:** LLMs struggle to accurately count line breaks and calculate statistical percentiles.
- **High Financial Cost:** Repeatedly prompting large tabular context is expensive and slow.

6.1 The Agentic Closed-Loop Pattern

`handler` serves as the perfect specialized tool for autonomous agents:



6.2 Sample Machine-Readable JSON Payload

```

{
  "file_path": "data.csv",
  "total_rows": 12,
  "total_columns": 6,
  "type_validation": {
    "total_mismatches": 2,
    "mismatch_details": [
      {
        "row": 3,
        "column": "age",
        "expected_type": "Integer",
        "found_value": "twenty-two",
        "reason": "Cannot parse as Integer"
      }
    ]
  },
  "outliers": {
    "total_outliers": 1,
    "outlier_details": [
      {
        "row": 10,
        "column": "salary",
        "value": 950000.0,
        "lower_bound": 34125.0,
        "upper_bound": 77125.0,
        "reason": "Exceeds upper bound 77125 (+82.7x IQR)"
      }
    ]
  }
}
  
```

```
}  
}
```

6.3 Python Agent Tool Implementation Example

Here is how easily a Python agent can drive `handler`:

```
import subprocess, json  
  
def audit_csv(file_path: str):  
    """Run handler audit and return structured JSON."""  
    result = subprocess.run(["handler", file_path, "--json"], capture_output=True, text=True)  
    return json.loads(result.stdout)  
  
def patch_csv(file_path: str, patch_spec: str):  
    """Patch a specific cell, e.g. '3:age=22'."""  
    subprocess.run(["handler", file_path, "--patch", patch_spec, "--in-place"], check=True)
```

7. Real-World Benchmark (1,000,000 Rows)

To demonstrate real-world production performance, `handler` was tested against a **1,000,000-row enterprise dataset** (`customers-1000000.csv` , 12 columns, ~150 MB).

Audit Task	Rows Processed	Execution Time	Key Discovery
Missing Values Scan	1,000,000	1.1 seconds	0 missing values (100% complete)
Type Inference & Validation	1,000,000	2.5 seconds	Inferred 1 Integer, 1 Date, 10 Text columns
Statistical Outlier Check	1,000,000	2.0 seconds	Index column within normal range
Email Collision Detection	1,000,000	2.1 seconds	Discovered 308 duplicate records across 307 email clusters!
Surgical Patch + 1M File Export	1,000,000	4.9 seconds	Patched Row 570 and wrote 1M rows cleanly

8. Command-Line Reference & Cheat Sheet

Command / Flag	Shorthand	Description
<code>handler <FILE></code>	-	Full health audit (Missing, Types, Outliers, Duplicates).
<code>--search_missing</code>	<code>-s</code>	Scan for empty cells, whitespace, and null tokens.
<code>--search_types</code>	<code>-t</code>	Audit for data type mismatches against inferred types.
<code>--search_outliers</code>	<code>-o</code>	Find statistical anomalies outside the 1.5 × IQR fences.
<code>--search_duplicates</code>	<code>-d</code>	Scan for byte-for-byte identical rows.
<code>--key <COL></code>	<code>-k</code>	Scan for duplicate collisions on a primary identifier (e.g. <code>--key Email</code>).
<code>--patch "ROW:COL=VAL"</code>	-	Surgically update a specific cell (e.g. <code>--patch "3:age=22"</code>).
<code>--drop-row <ROW_NUM></code>	-	Drop an individual row by index (e.g. <code>--drop-row 10</code>).
<code>--in-place</code>	-	Save edits directly to the input file atomically.
<code>--fix</code>	-	Trigger bulk auto-remediation (dedup, coerce, clean nulls).
<code>--fill-missing</code>	-	Impute missing numeric values with column medians.
<code>--drop-outliers</code>	-	Drop rows containing statistical outliers during repair.
<code>--output <PATH></code>	<code>-O</code>	Specify destination path for sanitized CSV.

Command / Flag	Shorthand	Description
<code>--json</code>	<code>-j</code>	Emit machine-readable JSON for scripts and AI agents.
<code>--verbose</code>	<code>-v</code>	Display all affected rows without the 10-row truncation.

HANDLER Technical Handbook • Open-Source MIT License • GitHub: [khokharsnehl45/handler](#)